DOKUMENTACJA TECHNICZNA • BEZPIECZEŃSTWO MICROSOFT GRAPH API

Przewodnik Bezpieczeństwa i Uwierzytelniania dla skryptu Upload-LargeFilesSession.ps1

Autor skryptu: Roman Pindela (v1.0.0) Opracowanie: Architektura Bezpieczeństwa & Hardening

Problem: Dlaczego podawanie -ClientSecret w linii poleceń jest niebezpieczne?

Wywołanie skryptu z jawnym parametrem `-ClientSecret "SecretValue..."` powoduje trwałe zapisanie tajnego klucza aplikacji Entra ID w pliku historii `PSReadLine` (`ConsoleHost_history.txt`), w dziennikach zdarzeń Windows (Event ID 4688 – tworzenie procesu z linią poleceń oraz Event ID 4104 – Script Block Logging), a także umożliwia odczyt klucza przez dowolny proces odpytujący listę procesów WMI (`Win32_Process`). Skrypt `Upload-LargeFilesSession.ps1` posiada jednak wbudowaną wielowarstwową architekturę obronną oraz natywną obsługę bezpiecznych metod uwierzytelniania.

1. Wbudowane mechanizmy obronne skryptu (Defensive Engineering)

ANALIZA KODU

Oprócz elastycznego podsystemu uwierzytelniania, sam kod skryptu realizuje szereg rygorystycznych praktyk bezpieczeństwa chroniących zarówno serwer lokalny, jak i docelowego tenanta Microsoft 365:

Mechanizm obronny	Lokalizacja w kodzie	Szczegółowy opis działania i znaczenie dla bezpieczeństwa
Sanityzacja wejścia i blokada wstrzyknięć	`Assert-SafeInputText` (linie 292–304)	Weryfikuje długość parametrów (maks. 512 znaków) oraz odrzuca znaki specjalne powłoki (`\0 ` ` $ < > | ; &`), eliminując ryzyko Command Injection.
Ochrona przed Path Traversal	Walidacja ścieżki (linie 328–332)	Normalizuje ukośniki i bezwzględnie blokuje sekwencje `..` oraz `//` w parametrze `-TargetFolder` , uniemożliwiając wyjście poza docelowy katalog OneDrive.
Pre-autoryzowane URL sesji (Zero-Token Chunking)	`Send-UploadSessionChunk` (linie 653–716)	Główny token `Bearer` służący do autoryzacji w Graph API **nie jest wysyłany** podczas przesyłania paczek danych (chunków). Metoda `PUT` trafia na tymczasowy, pre-autoryzowany adres `uploadUrl` .
Automatyczne sprzątanie porzuconych sesji	Blok `else` pętli (linie 865–871)	W przypadku błędu transferu skrypt wysyła żądanie `HTTP DELETE` na adres `uploadSessionUrl` , natychmiast unieważniając otwartą sesję po stronie serwerów Microsoft.
Odporność na Throttling (Exponential Backoff)	Obsługa błędów HTTP (linie 685–694)	Przechwytuje kody `HTTP 429` (Too Many Requests) oraz `5xx` , respektując nagłówek `Retry-After` zwracany przez Microsoft Graph, co chroni przed blokadą konta.

2. Metoda 1: Uwierzytelnianie Certyfikatem X.509

REKOMENDOWANY STANDARD ENTERPRISE

Najbezpieczniejszy sposób połączenia nienadzorowanego (np. z Harmonogramu Zadań Windows). Zamiast tekstowego hasła (`ClientSecret`) wykorzystuje parę kluczy kryptograficznych RSA 2048-bit. Klucz prywatny bezpiecznie spoczywa w lokalnym magazynie certyfikatów Windows (`Cert:\CurrentUser\My` lub `Cert:\LocalMachine\My`), a w Entra ID rejestrowany jest wyłącznie klucz publiczny (`.cer`). Skrypt obsługuje ten tryb natywnie w liniach **505–525** za pomocą parametru `-CertificateThumbprint` lub `-CertificatePath` .

KROK 1 **Wygenerowanie certyfikatu Self-Signed na serwerze wykonującym kopie (jednorazowo):**

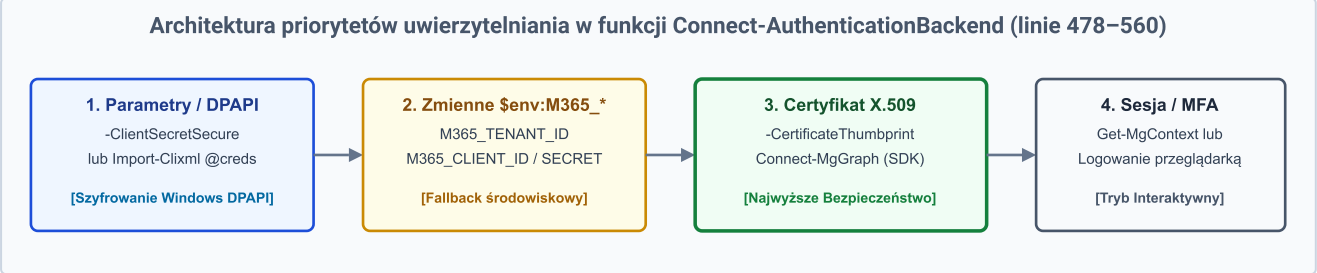
```
# Generowanie certyfikatu ważnego przez 2 lata w magazynie bieżącego użytkownika (konta usługowego)
$cert = New-SelfSignedCertificate -Subject "CN=OneDrive-Backup-Daemon" `
    -CertStoreLocation "Cert:\CurrentUser\My" `
    -KeyExportPolicy Exportable -KeySpec Signature `
```

KROK 2 **Rejestracja certyfikatu w portalu Microsoft Entra ID:**
Otwórz **Microsoft Entra admin center** (`entra.microsoft.com`) → **App registrations** → wybierz swoją aplikację → przejdź do sekcji **Certificates & secrets** → zakładka **Certificates** → kliknij **Upload certificate** i wskaż plik `C:\Temp\OneDrive-Backup.cer`. Po wgraniu certyfikatu możesz całkowicie usunąć stary `Client Secret`.

KROK 3 **Zapisanie identyfikatorów w zmiennych środowiskowych i czyste wywołanie skryptu:**
W liniach **480–481** skrypt automatycznie sprawdza zmienne środowiskowe `$env:M365_TENANT_ID` oraz `$env:M365_CLIENT_ID`, jeśli nie podano ich w parametrach wywołania. Zapisując je na stałe w profilu konta usługowego, całkowicie eliminujesz identyfikatory GUID z linii poleceń:

```
# Jednorazowy zapis identyfikatorów TenantId oraz ClientId w zmiennych środowiskowych użytkownika:
[Environment]::SetEnvironmentVariable("M365_TENANT_ID", "11111111-2222-3333-4444-555555555555", "User")
[Environment]::SetEnvironmentVariable("M365_CLIENT_ID", "aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee", "User")

# Docelowe, bezpieczne uruchomienie skryptu (bez haseł i bez GUID-ów w poleceniu):
.\Upload-LargeFilesSession.ps1 `
    -User "user@company.com" `
    -TargetFolder "SQL_Dumps" `
    -Path "E:\Dumps\*.bak" `
    -CertificateThumbprint "9A8B7C6D5E4F3A2B1C0D9E8F7A6B5C4D3E2F1A0B" `
    -ChunkSizeMB 25 `
    -LogPath "C:\Logs\OneDrive_Upload.log"
```



3. Metoda 2: Szyfrowanie DPAPI (Export-Clixml + -ClientSecretSecure) SZYBKIE WDROŻENIE

Jeśli nie chcesz generować certyfikatów, możesz użyć natywnego parametru `-ClientSecretSecure` (zdefiniowanego w linii **125** skryptu jako typ `[System.Security.SecureString]`) w połączeniu z mechanizmem **Windows DPAPI (Data Protection API)** oraz rozpakowywaniem parametrów (*PowerShell Splatting*). Plik utworzony przez `Export-Clixml` jest szyfrowany kluczem przypisanym ściśle do konta użytkownika Windows oraz konkretnej maszyny — skopiowanie pliku XML na inny komputer lub otwarcie go przez innego użytkownika uniemożliwi odszyfrowanie sekretu.

```
# KROK 1 (Jednorazowo na koncie wykonującym skrypt): Zapisanie zaszyfrowanej tablicy parametrów do pliku XML
$config = @{
    TenantId           = "11111111-2222-3333-4444-555555555555"
    ClientId           = "aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee"
    ClientSecretSecure = (Read-Host "Wklej Client Secret" -AsSecureString)
}
$config | Export-Clixml -Path "C:\Scripts\GraphCreds.xml"

# KROK 2 (W zadaniu produkcyjnym): Wczytanie pliku i przekazanie przez Splatting (@creds)
$creds = Import-Clixml -Path "C:\Scripts\GraphCreds.xml"
.\Upload-LargeFilesSession.ps1 @creds `
    -User "user@company.com" -TargetFolder "SQL_Dumps" `
    -Path "E:\Dumps\*.bak" -ChunkSizeMB 25 -LogPath "C:\Logs\OneDrive_Upload.log"
```

4. Metoda 3: Moduł Microsoft.PowerShell.SecretManagement LOKALNY SEJF

5. Metoda 4: Automatyczny odczyt DPAPI zaszyty w kodzie skryptu

MODYFIKACJA KODU

Jeżeli zależy Ci na tym, aby polecenie wywołujące skrypt zawierało **wyłącznie parametry biznesowe** (użytkownika, folder docelowy oraz ścieżkę do plików `.bak`) bez konieczności wywoływania `Import-Clixml` przed uruchomieniem skryptu, możesz rozbudować funkcję `Connect-AuthenticationBackend` (w okolicach linii **478** skryptu). Poniższa modyfikacja sprawia, że skrypt samoczynnie wykrywa i ładuje plik `GraphCreds.xml` znajdujący się w tym samym katalogu co skrypt:

```
function Connect-AuthenticationBackend {
    # [NOWOŚĆ] Automatyczny import poświadczeń z zaszyfrowanego pliku DPAPI (jeśli istnieje obok skryptu)
    $dpapiFile = Join-Path -Path $PSScriptRoot -ChildPath "GraphCreds.xml"
    if (Test-Path -Path $dpapiFile) {
        $savedAuth = Import-Clixml -Path $dpapiFile
        if ([string]::IsNullOrWhiteSpace($TenantId))   { $TenantId = $savedAuth.TenantId }
        if ([string]::IsNullOrWhiteSpace($ClientId))   { $ClientId = $savedAuth.ClientId }
        if (-not $ClientSecretSecure -and $savedAuth.ClientSecretSecure) {
            $ClientSecretSecure = $savedAuth.ClientSecretSecure
        }
    }
    # (... dalsza część oryginalnej funkcji Connect-AuthenticationBackend pozostaje bez zmian ...)
```

Po zastosowaniu powyższego rozszerzenia wywołanie produkcyjne upraszcza się do najczystszej postaci:

```
.\Upload-LargeFilesSession.ps1 -User "user@company.com" -TargetFolder "SQL_Dumps" -Path "E:\Dumps\*.bak" -ChunkSizeMB 25 -LogPath "C:\Logs\OneDrive_Upload.log"
```

6. Zestawienie porównawcze metod zabezpieczenia poświadczeń

MACIERZ DECYZYJNA

Metoda uwierzytelniania	Poziom bezp.	Zmiana w kodzie?	Charakterystyka i rekomendowane środowisko
0. Jawny -ClientSecret	**Krytycznie niski**	Brak	**Niezalecane.** Sekret widoczny w historii PSReadLine, logach procesów Windows (Event 4688/4104) oraz pamięci WMI.
1. Certyfikat X.509 (`-CertificateThumbprint`)	**Najwyższy (Enterprise)**	Nie (wbudowane, wymagany fix linii 566)	Całkowity brak haseł. Klucz prywatny chroniony w magazynie certyfikatów Windows. Idealne dla Harmonogramu Zadań i kont usługowych.
2. Plik DPAPI XML (`Export-Clixml` + `@creds`)	**Wysoki**	Nie (wbudowane `-ClientSecretSecure`)	Szyfrowane kluczem DPAPI konta użytkownika i maszyny. Najszybsze bezpieczne wdrożenie na pojedynczym serwerze bez generowania certyfikatów.
3. SecretManagement (`Microsoft.PowerShell.SecretStore`)	**Wysoki**	Nie	Centralny sejf poświadczeń na serwerze. Najlepszy wybór, gdy wiele różnych skryptów korzysta z tych samych poświadczeń Entra ID.
4. Zmienne środowiskowe (`$env:M365_CLIENT_SECRET`)	**Średni**	Nie (wbudowane w liniach 480–487)	Ukrywa sekret przed linią poleceń, ale przechowuje go w bloku środowiskowym